



Exploring Variations of Gradient Boosted Tweedie Models

Harri Vanhems

MATH 470 — Statistical Machine Learning

McGill University

April 6, 2025

Abstract

This report outlines the implementation of the Gradient Boosting algorithm into Tweedie-based models for insurance claim prediction, tracing developments from early boosting approaches to current zero-inflated extensions.

The main contribution of this report lies in the evaluation of new model variants that isolate individual sources of flexibility in the ZIT model. Through synthetic experiments, we show that estimating the mixture probability or dispersion non-parametrically often leads to worse performance due to overfitting. In contrast, keeping these components constant rather than as functions of covariates results in more stable and accurate models.

Contents

1	Introduction	3
2	Background	3
2.1	Tweedie and TDBoost Model Overview	3
2.2	Mathematics of Gradient Boosting	4
2.2.1	Decision Trees	5
2.2.2	Boosting Trees	6
2.3	EMTBoost Model Overview	7
2.4	ZIT Model Overview	9
3	Motivation for Model Modifications	9
4	Methodology and Results	10
4.1	Synthetic Data Experiments	10
4.1.1	Data Generation	11
4.1.2	Performance Metrics	11
4.2	Results	12
5	Conclusion and Future Work	13

1 Introduction

This report explores how different modeling strategies perform when predicting insurance claim amounts in the presence of extreme zero inflation. The main goal is to understand how increasing model flexibility—by allowing more components to depend on covariates impacts predictive accuracy. We focus on several variants of the Gradient Boosted Tweedie framework and evaluate their behavior through repeated experiments on synthetic data. By comparing these models using consistent metrics, we aim to pinpoint which design choices lead to real performance gains and which ones add unnecessary complexity.

2 Background

This section provides background on the key developments that led to the current model. It is divided into three subsections, each highlighting a new step that led to the model studied in Section 3.

2.1 Tweedie and TDBoost Model Overview

Modeling insurance claim data is challenging because the data tend to be non-negative, highly skewed, and contain a high concentration of zeros. This makes it hard for standard statistical distributions to capture patterns in the data. A common approach in the industry is to leverage **Tweedie models** to represent this unique data. Tweedie models obey the following log-density form:

$$\log f_Z(z \mid \mu, \phi, \rho) = \frac{1}{\phi} \left(z \frac{\mu^{1-\rho}}{1-\rho} - \frac{\mu^{2-\rho}}{2-\rho} \right) + \log a(z, \phi, \rho) \quad (1)$$

where the normalizing function $a(z, \phi, \rho)$ is defined as:

$$a(z, \phi, \rho) = \begin{cases} \frac{1}{z} \sum_{t=1}^{\infty} \frac{z^{t\alpha}}{(\rho-1)^{t\alpha} \phi^t (1+\alpha)(2-\rho)^t t! \Gamma(t\alpha)} & \text{for } z > 0 \\ 1 & \text{for } z = 0 \end{cases} \quad (2)$$

In the above, we refer to ρ as the **power parameter**, which we vary between 1 and 2. The parameter ϕ is the **dispersion parameter**, whose estimation is an essential part of this report. Finally, μ denotes the **mean** of the distribution.

Commonly, only μ is assumed to vary as a function of the covariates, while ϕ and ρ are estimated as constants through separate procedures. In fitting μ , the Tweedie model is often formulated within a **Generalized Linear Model (GLM)** framework, in which a restriction is placed on how μ can be modeled:

$$g(\mu) = \mathbf{x}^\top \boldsymbol{\beta} \quad (3)$$

In the above, $\mathbf{x}$, is the vector of covariates, that is, the vectors of features assigned to each policyholder, and g is a known log-link function. $\boldsymbol{\beta}$ is a vector of weights assigned to each element of $\mathbf{x}$.

Qian Y, Yang and J. Zou[1] proposed an alternate method in evaluating μ ; they removed the GLM setting and instead proposed the form: $g(\mu) = F(\mathbf{x})$, where g is as before and F is a function to be estimated non-parametrically, leveraging the tree based gradient boosting algorithm. Section 2.2 covers the Mathematics of the Gradient Boosting Algorithm.

To understand the mechanics of the TDBOOST algorithm, consider a portfolio of instances $(\mathbf{x}_i, y_i)$, where $i = 1, \dots, n$. The vector $\mathbf{x}_i$ corresponds to a set of features describing the i th policy contract, and y_i denotes the policy premium for the i th contract.

The log-likelihood of this model is given by:

$$\ell(\mu(\mathbf{x}_i), \phi, \rho \mid \{y_i, \mathbf{x}_i\}_{i=1}^n) = \sum_{i=1}^n \frac{1}{\phi} \left(y_i \frac{\mu(\mathbf{x}_i)^{1-\rho}}{1-\rho} - \frac{\mu(\mathbf{x}_i)^{2-\rho}}{2-\rho} \right) + \log a(y_i, \phi, \rho). \quad (4)$$

This log-likelihood serves as our loss function, whose gradient will be the target that the trees will be fitted to. We see that it requires an assumption on ρ , but not on ϕ . Hence, the TDBOOST algorithm works as follows: for each fixed ρ , the authors solve for the mean $\mu(\rho)$, and then maximize the log-likelihood with respect to ϕ to obtain $\phi(\rho)$. This procedure is repeated over a grid of ρ values to find the optimal ρ^* that maximizes the profile likelihood. Finally, the corresponding estimates $\mu(\rho^*)$ and $\phi(\rho^*)$ are returned.

2.2 Mathematics of Gradient Boosting

In this section, we go over the logic and mathematics of the gradient boosting algorithm. There is extensive literature available about the algorithm(notably in [2]); hence, only a brief overview will be given here. **Gradient Boosting** is an additive machine learning model sequentially built from multiple weaker models. In this research,

we choose **decision trees** as our weak learners. By *weak learners*, we mean relatively simple models that would perform poorly on their own but can be boosted to form an overall competent model when combined with other weak learners. Before defining Boosting, we define decision tree models clearly.

2.2.1 Decision Trees

A decision tree works by partitioning the training data into M regions $R_1, R_2, \dots, R_M$ (M is a hyperparameter of the model). Each region is assigned a constant c_m . Once the model is trained, its prediction for any input $\mathbf{x}$ is simply the constant c_m associated with the region into which $\mathbf{x}$ falls as it goes down the tree. To train a regression tree, that is, to find the best regions and best constants assigned to each region, one recursively finds the best feature and split point that minimizes the loss at each node, starting from the root.

If we minimize the sum of squared errors $\sum (y_i - f(x_i))^2$. The optimal prediction $\hat{c}_m$ for region R_m is the average of y_i in that region:

$$\hat{c}_m = \text{ave}(y_i \mid x_i \in R_m). \quad (9.11)$$

To find the best partition, we consider all possible splitting variables j and split points t , define:

$$R_1(j, t) = \{X \mid X_j \leq t\}, \quad R_2(j, t) = \{X \mid X_j > t\}. \quad (9.12)$$

We seek the pair (j, t) minimizing the total squared error:

$$\min_{j, t} \left[\min_{c_1} \sum_{x_i \in R_1(j, t)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, t)} (y_i - c_2)^2 \right]. \quad (9.13)$$

For fixed j and t , the optimal values are:

$$\hat{c}_1 = \text{ave}(y_i \mid x_i \in R_1(j, t)), \quad \hat{c}_2 = \text{ave}(y_i \mid x_i \in R_2(j, t)). \quad (9.14)$$

The process is then recursively applied to each resulting region until there are M sub-regions.

Figures 1 and 2 showcase visual representations of the data split procedure as well as how the actual decision process for a new datum would work.

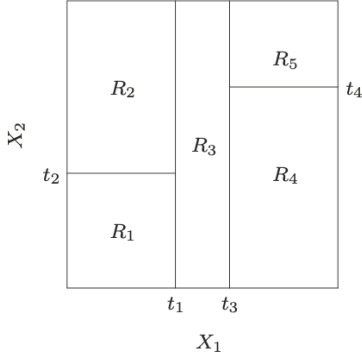


Figure 1: Example of a split dataset

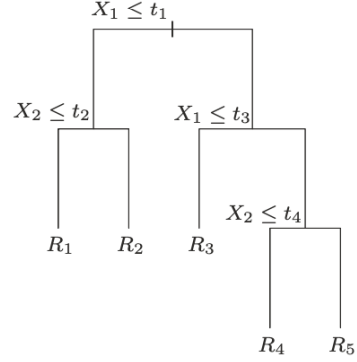


Figure 2: Example of a Regression Tree

We represent decision trees as $B(\mathbf{x}; \xi_m)$, where:

$$B(\mathbf{x}; \xi_m) = \sum_{j=1}^{J_m} c_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}), \quad m = 1, \dots, M. \quad (5)$$

Here, J_m is a hyperparameter corresponding to the number of terminal nodes in the m -th tree, $\mathbf{x}$ is the input feature vector. $\xi_m = (R_{jm}, c_{jm})$ contains the learned parameters of the tree where R_{jm} is the j -th region (leaf node) of the m -th decision tree, and c_{jm} is the constant value assigned to region R_{jm} .

2.2.2 Boosting Trees

Now that trees are defined, we make sense of Boosting, which means adding multiple *shallow* trees to make a better prediction. These trees will be added on sequentially where each new tree aims to correct the errors made by the previous ones. The key idea is to perform gradient descent in the function space. **The m -th tree will target the gradient of the loss between the actual value and the current model F_{m-1} indicating the direction in which the next model F_m should be adjusted.** Once the tree is fit, we assign it a weight γ_m by solving:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma B(\mathbf{x}_i; \xi_m)). \quad (6)$$

We then update the model by adding the new tree scaled by a shrinkage factor ν (a small positive number to avoid overfitting):

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \gamma_m B(\mathbf{x}; \boldsymbol{\xi}_m). \quad (7)$$

This iterative procedure is repeated over M iterations, resulting in the final model:

$$F(\mathbf{x}) = \sum_{m=1}^M \nu \gamma_m B(\mathbf{x}; \boldsymbol{\xi}_m). \quad (8)$$

The shrinkage factor ν controls the learning rate, thereby preventing overfitting by making each update more conservative. [2]

2.3 EMTBoost Model Overview

The TDBoost algorithm was one of the first models to showcase how Tweedie models can benefit from machine-learning assistance. However, they do not always yield optimal performance in situations where the empirical distribution is highly imbalanced and contains an excessively large proportion of zero claims. To address this, Wei Qian He Zhou and Yang[3] proposed a zero-inflated **mixture model** that combined a Tweedie distribution with probability q and an exact zero mass with probability $1 - q$:

$$Y = \begin{cases} Z, & \text{with probability } q, \quad \text{where } Z \sim \text{Tw}(\mu, \phi, \rho), \\ 0, & \text{with probability } 1 - q. \end{cases} \quad (9)$$

The point of this maneuver is to relieve some of the weight from the Z variable so that it can better model the rest of the distribution by being assigned only a portion of the zeros. As before, to optimize this model, we aim to minimize the negative log-likelihood, as we did with the TDBoost algorithm. However, the mixture model makes this task computationally much more difficult to carry out directly. To address this, the authors proposed a **Gradient Boosted Expectation-Maximization (EM)** algorithm to find the optimal model parameters $\hat{\theta} = (\hat{F}, \hat{\phi}, \hat{\rho})$.

The construction follows these steps. Since it is unknown whether a zero arises from the exact point mass or from the Tweedie distribution, we introduce a **latent variable** to represent this assignment.

We introduce n independent Bernoulli latent variables $\Pi_1, \dots, \Pi_n$ such that for $i = 1, \dots, n$, $P(\Pi_i = 1) = q$, and $\Pi_i = 1$ when y_i is sampled from $\text{Tweedie}(\mu_i, \phi, \rho)$, and $\Pi_i = 0$ if y_i is from the exact zero point mass. The EM algorithm is a two-step iterative process used to estimate parameters when there are latent variables

(discussed extensively in [4]). In the E-step, we compute the expected values of the latent variables Π_i , given the current parameter estimates (call the parameters $\hat{\theta}^t = (\hat{F}^t, \hat{\phi}^t, \hat{q}^t)$ at the t -th iteration). This gives us a soft assignment of whether each y_i came from the Tweedie part or the exact zero part. In the M-step, we update the parameters $\hat{\theta}$ by maximizing $\mathbf{Q}(\theta | \theta^t)$: the expected complete-data log-likelihood computed in the E-step. These two steps are repeated until convergence. To quantify this statement:

For the **E-step** update:

The posterior distribution of each latent variable Π_i is

$$f(\Pi_i | y_i, \theta) = \frac{f(y_i, \Pi_i | \theta)}{f(y_i, \Pi_i | \theta) + f(y_i, 1 - \Pi_i | \theta)} \quad (10)$$

In Equation 10, $f(y_i, \Pi_i | \theta)$ is the joint distribution of the complete model defined as:

$$f(y_i, \Pi_i | \theta) := (q \cdot f_Z(y_i | g^{-1}(F(\mathbf{x}_i)), \phi))^{\Pi_i} ((1 - q) \cdot \mathbb{I}\{y_i = 0\})^{1 - \Pi_i} \quad (11)$$

Using this definition we can compute Equation 11 at $\theta = \hat{\theta}^t$ for $\Pi_i = 0, 1$ and plug into the expected complete-data log-likelihood to get:

$$\mathbf{Q}(\theta | \theta^t) = \sum_{i=1}^n [f(\Pi_i = 1 | y_i, \theta^t) \log f(y_i, \Pi_i = 1 | \theta) + f(\Pi_i = 0 | y_i, \theta^t) \log f(y_i, \Pi_i = 0 | \theta)] \quad (12)$$

For the **M-step** update:

Having updated our $\mathbf{Q}$ function with the newest parameters in Equation 12, we can now find the new best parameters with respect to this updated $\mathbf{Q}$.

$$\theta^{t+1} = (F^{t+1}, \phi^{t+1}, q^{t+1}) \leftarrow \arg \max_{\theta \in \Theta} \mathbf{Q}(\theta | \theta^t) \quad (13)$$

in which F^{t+1} , ϕ^{t+1} , and q^{t+1} are updated successively:

$$F^{t+1} \leftarrow \arg \max_{F \in \mathcal{F}} \mathbf{Q}(F | (F^t, \phi^t, q^t)) \quad (14)$$

$$\phi^{t+1} \leftarrow \arg \max_{\phi \in \mathbb{R}^+} \mathbf{Q}(\phi | (F^{t+1}, \phi^t, q^t)) \quad (15)$$

$$q^{t+1} \leftarrow \arg \max_{q \in [0,1]} \mathbf{Q}(q | (F^{t+1}, \phi^{t+1}, q^t)) \quad (16)$$

The procedure in Equation 14 is conducted using the Gradient Boosting Algorithm we discussed in Section 2.2.2 for TDBOOST.

2.4 ZIT Model Overview

Researchers Yuwen Gu and Zijian Huang[5], were motivated to further improve EMTBoost, seeing as it had largely improved upon its predecessor TDBOOST by now focusing on two of the parameters held fixed in EMTBoost: The dispersion ϕ and the mixture probability q . The improvement ZIT offers, is that it no longer assumes a constant ϕ and q across different contracts and policyholders, but rather assumes a dependence on the covariates, that is: $\phi = \phi(\mathbf{x}_i)$ and $q = q(\mathbf{x}_i)$. Consequently, we now seek to find (F_μ, F_ϕ, F_q) , three non-parametric functions to be estimated with Gradient Boosting. Note that (μ, ϕ, q) are related to their respective function through a known log-link. These two changes don't impact the architecture of the algorithm, rather, they only change the optimization process we defined in Equation 13. The M-step in the ZIT model's EM algorithm is now:

$$\boldsymbol{\theta}^{t+1} = (F_\mu^{t+1}, F_\phi^{t+1}, F_q^{t+1}) \leftarrow \arg \max_{\boldsymbol{\theta} \in \Theta} \mathbf{Q}(\boldsymbol{\theta} \mid \boldsymbol{\theta}^t) \quad (17)$$

in which F_μ^{t+1} , F_ϕ^{t+1} , and F_q^{t+1} are updated successively using the Gradient Boosting Algorithm:

$$F_\mu^{t+1} \leftarrow \arg \max_{F_\mu \in \mathcal{F}} \mathbf{Q}(F_\mu \mid (F_\mu^t, F_\phi^t, F_q^t)) \quad (18)$$

$$F_\phi^{t+1} \leftarrow \arg \max_{F_\phi \in \mathcal{F}} \mathbf{Q}(F_\phi \mid (F_\mu^{t+1}, F_\phi^t, F_q^t)) \quad (19)$$

$$F_q^{t+1} \leftarrow \arg \max_{F_q \in \mathcal{F}} \mathbf{Q}(F_q \mid (F_\mu^{t+1}, F_\phi^{t+1}, F_q^t)) \quad (20)$$

Exactly as in Section 2.3, the E-M steps are repeated until model convergence in which case the optimal non-parametric functions are returned. Furthermore, though it was not discussed in Section 2.3. Upon ending of the EM algorithms for the EMT and ZIT models, a profile likelihood was run on ρ to further improve the model.

3 Motivation for Model Modifications

Our initial experiments showed that directly moving from the simpler EMTBoost model, which treats only the mean function $\mu(x)$ nonparametrically, to the fully flexible ZIT model—which treats three different components $\mu(x)$, $\phi(x)$, and $q(x)$ nonparametrically—resulted in unexpected underperformance. This suggested that the full ZIT approach might be “overly flexible” or difficult to train effectively in practice.

To investigate where this underperformance originates, we developed a sequence of “middle-ground” models that gradually introduce additional nonparametric components. Specifically, we wanted to see whether adding just one extra nonparametric component (either the zero-inflation probability q or the dispersion ϕ) would still lead to the same performance issue, or if this problem arose only when all three components were estimated simultaneously.

In particular, we constructed three intermediate model variants:

- **ZIT0:** Here, we estimate the dispersion parameter ϕ *outside* the EM loop via profile likelihood, treating it as a constant of the covariates. Meanwhile, both $\mu(x)$ and $q(x)$ are allowed to vary with the covariates, and are fitted nonparametrically using gradient boosting.
- **ZIT1:** Similar to ZIT0, except ϕ is *optimized inside* the EM loop while still treated as a constant of the covariates. We again allow $\mu(x)$ and $q(x)$ to be estimated nonparametrically.
- **ZIT2:** In this variant, we fix q as a single constant optimized inside the EM loop and only estimate $\mu(x)$ and $\phi(x)$ via gradient boosting. This means $\phi(x)$ is now modeled nonparametrically, whereas q is not.

Each variant isolates the effect of adding exactly one additional nonparametric component beyond what EMTBoost already employs. By comparing performance across these three intermediate models and EMTBoost itself, we can more clearly identify which part of the ZIT model introduces the greatest instability.

4 Methodology and Results

The goal of this experiment is to compare different modeling approaches, in particular, it will be to see how our model variants introduced in Section 3 perform against the existing models described in Section 2

In the R implementation, each model is trained using gradient tree boosting via the LightGBM framework and evaluated through three performance metrics, which are discussed in Section 4.1.2.

4.1 Synthetic Data Experiments

To evaluate the performance of our proposed (ZIT0), (ZIT1) and (ZIT2) models against TDBOost, EMTBoost and ZIT, we conducted experiments using synthe-

cally generated data. In section 4.1.1, we describe the data generation process and experimental design in detail.

4.1.1 Data Generation

We simulate a dataset of size $N = 8000$, partitioned into 3000 training samples and 5000 test samples. Each observation consists of a feature vector $x_i \in \mathbb{R}^{20}$, with entries drawn i.i.d. from a standard normal distribution:

$$X \in \mathbb{R}^{N \times 20}, \quad x_{ij} \sim \mathcal{N}(0, 1).$$

To generate the response variable Y_i , we construct three non-linear functions $f_\pi(x)$, $f_\mu(x)$, and $f_\phi(x)$ using a random basis generator with 20 components. These functions define the **true underlying model parameters**: The response Y_i is then drawn from a zero-inflated Tweedie model with power $\rho = 1.5$:

$$Y_i = \begin{cases} 0, & \text{with probability } \pi(x_i), \\ \text{Tweedie}(\mu(x_i), \phi(x_i), \rho), & \text{with probability } 1 - \pi(x_i). \end{cases}$$

The true functions $\mu(x)$, $\phi(x)$, and $\pi(x)$ are retained for evaluating the accuracy of fitted models.

4.1.2 Performance Metrics

To evaluate model performance, we consider three common metrics: Mean Squared Error (MSE), Mean Absolute Deviation (MAD), and the Gini coefficient. Here, we denote the true mean we generated synthetically as $\mu(x)$ and the predicted nonparametric mean from a given model as $\hat{\mu}_{\text{pred}}(x)$.

Mean Squared Error (MSE) is defined as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\mu(x_i) - \hat{\mu}_{\text{pred}}(x_i))^2$$

It penalizes larger errors more heavily due to squaring, making it sensitive to outliers. A lower MSE indicates better predictive accuracy.

Mean Absolute Deviation (MAD) is given by:

$$\text{MAD} = \frac{1}{n} \sum_{i=1}^n |\mu(x_i) - \hat{\mu}_{\text{pred}}(x_i)|$$

MAD is more robust to outliers than MSE and provides a more interpretable measure of average prediction error.

Gini coefficient given by:

$$\text{Gini} = \frac{\text{Cov}(\mu(x), \text{rank}(\hat{\mu}_{\text{pred}}(x)))}{\text{Cov}(\mu(x), \text{rank}(\mu(x)))} \quad (21)$$

In applications like insurance, where the goal is to identify high-risk individuals, it's more important to rank policyholders correctly than to predict exact amounts. **The Gini coefficient tells you how well your model ranks people by risk, not whether the predicted values (e.g. suggested premiums) are numerically accurate.**

A Gini coefficient close to 1 indicates excellent separation between high and low outcomes, while a value near 0 suggests poor ranking performance.

4.2 Results

Table 1 summarizes the performance of each model based on 12 independent simulation runs. In each run, new data was generated randomly, and we evaluated the accuracy of each model using the metrics idscussed in section 4.1.2.

To enable fair comparison across runs with different scales, we normalized MAD and MSE within each run by dividing all values by the corresponding value for the baseline model (TD). GINI, being inherently scale-free, was left unchanged. The table shows the mean and error of each metric across the 12 runs.

Table 1: Average normalized performance metrics over 12 runs.

Model	MAD	MSE	GINI
TD	1.000 ± 0.000	1.000 ± 0.000	0.160 ± 0.081
EMT	0.302 ± 0.063	0.136 ± 0.044	0.165 ± 0.075
ZIT	0.478 ± 0.177	0.313 ± 0.183	0.169 ± 0.070
ZIT0	0.312 ± 0.068	0.138 ± 0.045	0.166 ± 0.072
ZIT1	0.326 ± 0.080	0.146 ± 0.055	0.164 ± 0.074
ZIT2	0.493 ± 0.179	0.320 ± 0.185	0.171 ± 0.068

From the results, several conclusions can be drawn. We first discuss two expected results worth mentioning. First, we clearly see that introducing the EM algorithm in EMT significantly improves the basic TD model by better accounting for the excess zeros in the data which confirms results in the article introducing EMTBoost[3].

Second, we do see that the base ZIT model under-performs as expected, which motivated the development of model variants.

We now draw three key insights about our model variants from the data.

First, the comparison between EMT and ZIT2 suggests that modeling the dispersion parameter $\phi(\mathbf{x}_i)$ non-parametrically does not improve performance. In fact, ZIT2 performs similarly to the underperforming ZIT model. This indicates that the dispersion parameters should remain constant across covariates.

Second, EMT, which treats ϕ as constant, performs comparably — or slightly better — than ZIT0 and ZIT1, both of which estimate $q(x)$ non-parametrically. This suggests that there is no significant gain from modeling the probability coefficient as a function of covariates. The added flexibility is likely outweighed by increased variance and risk of overfitting.

Third, the nearly identical performance of ZIT0 and ZIT1 indicates that whether ϕ is estimated via profile likelihood (outside the EM loop) or within the loop has minimal impact. This suggests that simpler estimation procedures may suffice without compromising predictive performance.

5 Conclusion and Future Work

Our investigation into model variants shows that added flexibility does not always translate into better performance. While the fully non-parametric ZIT model aimed to improve predictive power by allowing all components to vary with the input features, it often underperformed due to increased variance and overfitting.

By isolating each component, we found that keeping the mixture probability and dispersion parameter constant generally led to more stable and accurate results. In particular, estimating the dispersion parameter non-parametrically did not offer any improvement and often degraded performance. Similarly, allowing the probability coefficient to vary across observations did not yield noticeable gains compared to treating it as a constant.

These results suggest that a simpler model—like EMTBoost—strikes a better balance between flexibility and stability.

Future work could explore the use of regularization techniques in the ZIT model

and its variants, especially since their underperformance may be linked to overfitting. It would also be worth revisiting the current R implementation, as some of the results may have been affected by tuning limitations or computational inefficiencies. Finally, applying these models to real-world insurance datasets would help confirm whether the insights from our synthetic experiments hold in practice.

References

- [1] J. Qian Y. Yang and J. Zou. Insurance premium prediction via gradient tree-boosted tweedie compound poisson models. *Journal of Business & Economic Statistics*, 2018.
- [2] Robert Tibshirani Jerome H. Friedman and Trevor Hastie. *The Elements of Statistical Learning*. Springer, 2017.
- [3] Wei Qian He Zhou and Yi Yang. Tweedie gradient boosting for extremely unbalanced zero-inflated data.
- [4] Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [5] Zijian Huang Yuwen Gu. Dispersion modeling in zero-inflated tweedie models with applications to insurance claim data analysis. *Draft Paper, not yet published*, 2025.